Service Portal

KB0042096

Using Slurm on ICE

Using Slurm on ICE

Accessing Computational Resources via Jobs

Request resources from the scheduler to be assigned space on a compute node. For all types of job submissions, the scheduler will assign space to you when it becomes available. Batch and interactive jobs both wait in the same queues for available space.



Tip

Visit our conversion guide to convert your PBS scripts from prior to May 2023 to Slurm scripts.



Tip

For graphical interactive jobs, including Jupyter notebooks, use Open OnDemand.

View this very useful guide from SchedMD for additional Slurm commands and options beyond those listed below. Further guidelines on more advanced scripts are in the user documentation on this page. The sections below are covered in detail on this page, click on link to navigate:

1. Informational Commands
2. Job Submission
3. Job Submission Examples

- Interactive Jobs
- Batch Jobs - Basic Python Example
- Choosing a CPU Architecture
- MPI Jobs
- GPU Jobs
- Local Disk Jobs

Informational Commands

squeue

Use `squeue` to check job status for pending (PD) and running (R) jobs. Many options are available to include with the command, including these:

- Add `-j <job number>` to show information about specific jobs. Separate multiple job numbers with a comma.
- Add `-u <username>` to show jobs belonging to a specific user, e.g., `-u gburdell3` .
- Add `-p <partition>` to see jobs submitted to a specific partition, e.g., `-p ice-cpu` .
- Add `-q <QOS>` to see jobs submitted to a specific QOS, e.g., `-q coc-grade` .
- Run `man squeue` or visit the squeue documentation page for more options.

sacct

After a job has completed, use `sacct` to find information about it. Many of the same options for `squeue` are available.

- Add `-j <job number>` to find information about specific jobs.
- Add `-u <username>` to see all jobs belonging to a specific user.
- Add `-X` to show information only about the allocation, rather then steps inside it.
- Add `-S <time>` to list jobs only after a specified time. Multiple time formats are accepted, including YYYY-MM-DD[HH:MM[:SS]], e.g., 2022-08-0119:05:23.
- Add `-o <fields>` to specify which columns of data should appear in the output. Run `squeue --helpformat` to see a list of available fields.
- Run `man sacct` or visit the sacct documentation page for more options.

scancel

To cancel a job, run `scancel <job number>` , e.g., `scancel 1440` to cancel job 1440. You can use `squeue` to find the job number first.

pace-check-queue

The `pace-check-queue` utility provides an overview of current utilization of each partition's nodes. Use the name of a specific partition as the input, e.g., `pace-check-queue ice-cpu`. On Slurm clusters, utilized and allocated local disk (including percent utilization) are not available.

- Add `-s` to see all features of each node in the partition.
- Add `-c` to color-code the "Accepting Jobs?" column.

pace-job-summary

The `pace-job-summary` provides high level overview about job processed on the cluster. Usage of the utility is very simple as follows:

```
$ pace-job-summary
Usage: `pace-job-summary <JobID>`
```

Output example:

```
$ pace-job-summary 2836
---------------------------------------
Begin Slurm Job Summary for 2836
Query Executed on 2022-08-17 at 18:21:33
---------------------------------------
Job ID:     2836
User ID:    gburdell3
Job name:   SlurmPythonExample
Resources:  cpu=4,mem=4G,node=1
Rsrc Used:  cput=00:00:08,vmem=0.8M,walltime=00:00:02,mem=0.0M,energy_used=0
Exit Code:  0:0
Partition:  ice-cpu
Nodes:      atl1-0-00-000-0-0
---------------------------------------
Batch Script for 2836
---------------------------------------
#!/bin/bash
#SBATCH -JSlurmPythonExample             # Job name
#SBATCH -N1 -n4                          # Number of nodes and cores required
#SBATCH --mem-per-cpu=1G                 # Memory per core
#SBATCH -t15                             # Duration of the job (Ex: 15 mins)
#SBATCH -oReport-%j.out                  # Combined output and error messages fil
e
#SBATCH --mail-type=BEGIN,END,FAIL       # Mail preferences
#SBATCH --mail-user=gburdell3@gatech.edu # E-mail address for notifications
cd $SLURM_SUBMIT_DIR                     # Change to working directory

module load anaconda3                    # Load module dependencies
srun python test.py                      # Example Process
---------------------------------------
```

Job Submission

Each job can request a maximum of 512 CPU hours and 16 GPU hours. The maximum walltime is 18 hours for CPU jobs and 16 hours for GPU jobs, unless further restricted by the CPU hour or GPU hour maximum.

Jobs that do not include a resource request will receive 1 core and 1 GB of memory/core for 1 hour.

Assignment of partitions and QOSs is generally handled automatically on ICE, so there's no need to specify them in most cases. All nodes are accessible to all students on ICE, and priority for different subsets of nodes is handled behind the scenes.

(Optional) Students in multiple courses

Jobs submitted by students enrolled in multiple ICE courses from a combination of colleges will default to a specific one. A student enrolled in a CoE course will default to prioritizing CoE nodes; a student enrolled in both CoC and other non-CoE college courses will default to prioritizing CoC nodes. All ICE nodes are always accessible to all students. Optionally, you can add `-q pace-ice` to your sbatch or salloc directives to prioritize non-CoC/CoE nodes for work associated with a non-CoC/CoE course. Adding `-q coe-ice` will maintain priority for CoE nodes, while adding `-q coc-ice` will set CoC priority for students enrolled in both CoE and CoC courses (or maintain priority for students enrolled in both a CoC course and a non-CoC/CoE course). Use of these flags is optional.

Grading Priority

Instructors and TAs have access to a high-priority QOS for grading assignments. For CoC courses, add `-q coc-grade` to sbatch or salloc directives. For CoE courses, add `-q coe-grade` to sbatch or salloc directives. For other courses, add `-q pace-grade`. Unlike ordinary jobs, these jobs can run for 24 hours of walltime, requesting a maximum of 768 CPU hours and 24 GPU hours each. Each instructor or TA can submit up to 10 jobs at a time to the grading QOS.

Job Submission Examples

Interactive Jobs

Interactive jobs allow interactive use, so you can work "live" and provide additional input as your computations run. Please use interactive jobs instead of the login nodes for intensive computations. ICE offers both **command-line interactive jobs** and graphical interactive jobs with **Open OnDemand** (including **Jupyter**). Graphical interactive jobs are required if you need a graphical user interface (GUI).

We recommend using the `salloc` to allocate resources for a command-line interactive job to work on the command line on a compute node. This is ideal for avoiding overuse of the login node while compiling and running test codes.

The number of nodes (--nodes or -N), CPU cores (--ntasks-per-node for cores per node or -n for total cores), and wall time requested (--time or -t using the format D-HH:MM:SS for days, hours, minutes, and seconds) may be designated. Run `man salloc` or visit the salloc documentation page for more options.

In this example, use `salloc` to allocate 1 node with 4 cores for an interactive job:

```
$ salloc -N1 --ntasks-per-node=4 -t1:00:00
salloc: Pending job allocation 1464
salloc: job 1464 queued and waiting for resources
```

After pending status, your job will start after resources are granted with the following prompt:

```
$ salloc -N1 --ntasks-per-node=4 -t1:00:00
salloc: Granted job allocation 1464
salloc: Waiting for resource configuration
salloc: Nodes atl1-1-02-007-30-2 are ready for job
----------------------------------------
Begin Slurm Prolog: Oct-07-2022 16:10:49
Job ID:    1464
User ID:   gburdell3
Job name:  interactive
Partition: ice-cpu
----------------------------------------
[gburdell3@atl0 ~]$
```

Once resources are available for the job, you should be automatically logged into an interactive job on a compute node with the resources requested from the login node. Here, in this interactive session, use `srun` with `hostname`:

```
[gburdell3@atl0 ~]$ srun hostname
atl0.pace.gatech.edu
atl0.pace.gatech.edu
atl0.pace.gatech.edu
atl0.pace.gatech.edu
```

Note that there are 4 instances of the node hostname because we requested 1 node with 4 cores. To exit the interactive job, you can wait for the allotted time to expire in your session (in this example, 1 hour) or you can exit manually using `exit`:

```
[gburdell3@atl0 ~]$ exit
exit
salloc: Relinquishing job allocation 1464
salloc: Job allocation 1464 has been revoked.
```

Batch Jobs

Batch jobs are for "submit and forget" workflows. Batch jobs are ideal for larger (many CPU) and longer (many hour) computations.

Write a Slurm script as a plain text file, then submit it with the `sbatch` command. **Any computationally-intensive command should be prefixed with `srun` for best performance using Slurm.**

- On PACE, you can use a text editor such as `nano`, `vi`, or `emacs` to create a plain text file. For beginners, `nano` is recommended. Type the command `nano` to launch it. Type `nano <filename>` to open an existing file or create a new one.

- (Required) Start the script with `#!/bin/bash` .
- Name a job with `#SBATCH –J <job name>` .
- Include resource requests:
    - For requesting cores, we recommend 1 of 2 options:
        1. `#SBATCH –n` or `#SBATCH ––ntasks` specifies the number of cores for the entire job. **The default is 1 core.**
        2. `#SBATCH –N` specifies the number of nodes, combined with `#SBATCH ––ntasks–per–node` , which specifies the number of cores per node.
    - For requesting memory, we recommend 1 of 2 options:
        1. For CPU-only jobs, use `#SBATCH ––mem–per–cpu=<request with units>` , which specifies the amount of memory per core. To request all the memory on a node, include `#SBATCH ––mem=0` . **The default is 1 GB/core.**
        2. For GPU jobs, you can instead use `#SBATCH ––mem–per–gpu=<request with units>` , which specifies the amount of memory per GPU.
- Request walltime with `#SBATCH –t` . Job walltime requests (`#SBATCH –t`) should use the format D-HH:MM:SS for days, hours, minutes, and seconds requested. Alternatively, include just an integer that represents minutes. **The default is 1 hour.**
- Name your output file, which will include both STDOUT and STDERR, with `#SBATCH –o <file name>` .
- If you would like to receive email notifications, include `#SBATCH ––mail–type=NONE,BEGIN,END,FAIL,ARRAY_TASKS,ALL` with only the conditions you prefer.
    - If you wish to use a non-default email address, add `#SBATCH ––mail–user=<preferred email>` .
- When listing commands to run inside the job, any computationally-intensive command should be prefixed with `srun` for best performance.
- Run `man sbatch` or visit the sbatch documentation page for more options.

Basic Python Example

- The guide will focus on providing a full runthrough of loading software and submitting a job
- In this guide, we'll load `anaconda3` and run a simple python script

While logged into ICE, use a text editor such as `nano` , `vi` , or `emacs` to create the following python script, call it `test.py`

```
#simple test script
result = 2 ** 2
print("Result of 2 ^ 2: {}".format(result))
```

Now, create a job submission script `SlurmPythonExample.sbatch` with the commands below:

```
#!/bin/bash
#SBATCH -JSlurmPythonExample             # Job name
#SBATCH -N1 --ntasks-per-node=4          # Number of nodes and cores per node req
uired
#SBATCH --mem-per-cpu=1G                 # Memory per core
#SBATCH -t15                             # Duration of the job (Ex: 15 mins)
#SBATCH -oReport-%j.out                  # Combined output and error messages fil
e
#SBATCH --mail-type=BEGIN,END,FAIL       # Mail preferences
#SBATCH --mail-user=gburdell3@gatech.edu # E-mail address for notifications
cd $SLURM_SUBMIT_DIR                     # Change to working directory

module load anaconda3                    # Load module dependencies
srun python test.py                      # Example Process
```

- Make sure that `test.py` and `SlurmPythonExample.sbatch` are in the same folder. It is important that you submit the job from this directory. `$SLURM_SUBMIT_DIR` is a variable that contains path for this directory where job is submitted.
- `module load anaconda3` loads anaconda3, which includes python.
- `srun python test.py` runs the python script. `srun` runs the program as many times as specified by the `-n` or `--ntasks` option. If we have just `python test.py` , then the program will run only once.

You can submit the script by running `sbatch SlurmPythonExample.sbatch` from command line. For checking job status, use `squeue -u gburdell3` . For deleting a job, use `scancel <jobid>` . Once the job is completed, you'll see a `Report-<jobid>.out` file, which contains the results of the job. It will look something like this:

```
#Output file
----------------------------------------
Begin Slurm Prolog: Oct-07-2022 16:10:04
Job ID:    1470
User ID:   gburdell3
Job name:  SlurmPythonExample
Partition: ice-cpu
----------------------------------------
Result of 2 ^ 2: 4
Result of 2 ^ 2: 4
Result of 2 ^ 2: 4
Result of 2 ^ 2: 4
----------------------------------------
Begin Slurm Epilog: Oct-07-2022 16:10:06
Job ID:        1470
Array Job ID:  _4294967294
User ID:       gburdell3
Job name:      SlurmPythonExample
Resources:     cpu=4,mem=4G,node=1
Rsrc Used:     cput=00:00:12,vmem=8K,walltime=00:00:03,mem=0,energy_used=0
Partition:     ice-cpu
Nodes:         atl0
----------------------------------------
```

Choosing a CPU Architecture

The cluster provides nodes with either Intel or AMD CPUs. By default, jobs are assigned to the first available resource.

- To request a node with an Intel CPU, add `#SBATCH –C intel`.
- To request a node with an AMD CPU, add `#SBATCH –C amd`.
- To request a node with an Intel Granite Rapids CPU, add `#SBATCH –C graniterapids`.

MPI Jobs

Warning

Do not use `mpirun` or `mpiexec` with Slurm. Use `srun` instead.

You may want to run Message Passing Interface (MPI) jobs, which utilize a message-passing standard designed for parallel computing on the cluster.

In this set of examples, we will compile "hello world" MPI code from MPI Tutorial and run the program using `srun`.

To set up our environment for both MPI job examples, follow the following steps to create a new directory and download the MPI code:

```
$ mkdir slurm_mpi_example
$ cd slurm_mpi_example
$ wget https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutoria
ls/mpi-hello-world/code/mpi_hello_world.c
```

Interactive MPI Example

For running MPI in Slurm using an interactive job, follow the steps for Interactive Jobs to enter an interactive session:

- First, as in the interactive job example, use `salloc` to allocate 1 node with 4 cores for an interactive job:

```
$ salloc –N2 ––ntasks-per-node=4 –t1:00:00
salloc: Pending job allocation 1471
salloc: job 1471 queued and waiting for resources
```

- Next, after pending status, your job will start after resources are granted with the following prompt:

```
salloc: job 1902 has been allocated resources
salloc: Granted job allocation 1471
salloc: Waiting for resource configuration
salloc: Nodes atl0,atl1 are ready for job
----------------------------------------
Begin Slurm Prolog: Oct-07-2022 16:10:09
Job ID:    1471
User ID:   gburdell3
Job name:  interactive
Partition: ice-cpu
----------------------------------------
[gburdell3@atl0 ~]$
```

- Next, within your interactive session and in the `slurm_mpi_example` directory created earlier with the `mpi_hello_world.c` example code, load the relevant modules and compile the MPI code using `mpicc` :

```
$ cd slurm_mpi_example
$ module load gcc mvapich2
$ mpicc mpi_hello_world.c –o mpi_hello_world
```

- Next run the MPI job using `srun` :

```
$ srun mpi_hello_world
```

- Finally, the following should be output from this interactive MPI example:

```
Hello world from processor atl0, rank 0 out of 8 processors
Hello world from processor atl0, rank 2 out of 8 processors
Hello world from processor atl0, rank 3 out of 8 processors
Hello world from processor atl1, rank 4 out of 8 processors
Hello world from processor atl1, rank 7 out of 8 processors
Hello world from processor atl0, rank 1 out of 8 processors
Hello world from processor atl1, rank 5 out of 8 processors
Hello world from processor atl1, rank 6 out of 8 processors
```

Batch MPI Example

For running MPI in Slurm using a batch job, follow the steps in Batch Jobs and Basic Python Example to set up and run a batch job.

- First, in the `slurm_mpi_example` directory created earlier with the `mpi_hello_world.c` example code, create a file named `SlurmBatchMPIExample.sbatch` with the following content:

```
#!/bin/bash
#SBATCH -JSlurmBatchMPIExample           # Job name
#SBATCH -N2 --ntasks-per-node=4          # Number of nodes and cores per node req
uired
#SBATCH --mem-per-cpu=1G                 # Memory per core
#SBATCH -t1:00:00                        # Duration of the job (Ex: 1 hour)
#SBATCH -oReport-%j.out                  # Combined output and error messages fil
e
#SBATCH --mail-type=BEGIN,END,FAIL       # Mail preferences
#SBATCH --mail-user=gburdell3@gatech.edu # E-mail address for notifications

cd $HOME/slurm_mpi_example               # Change to working directory created in
$HOME

# Compile MPI Code
module load gcc mvapich2
mpicc mpi_hello_world.c -o mpi_hello_world

# Run MPI Code
srun mpi_hello_world
```

- This batch file combines the configuration for the Slurm batch job submission, the compilation for the MPI code, and running the MPI job using `srun` .
- Next run the MPI batch job using `sbatch` in the `slurm_mpi_example` directory:

```
$ cd slurm_mpi_example
$ sbatch SlurmBatchMPIExample.sbatch
Submitted batch job 1473
```

- This example should not take long, but it may take time to run depending on how busy the cluster is.
- Finally, after the batch MPI job example has run, the following should be output in the file created in the same directory named `Report-<job id>.out` :

```
------------------------------------
Begin Slurm Prolog: Oct-07-2022 16:10:09
Job ID:    1473
User ID:   gburdell3
Job name:  SlurmBatchMPIExample
Partition: ice-cpu
------------------------------------
Hello world from processor atl0, rank 0 out of 8 processors
Hello world from processor atl0, rank 2 out of 8 processors
Hello world from processor atl0, rank 3 out of 8 processors
Hello world from processor atl1, rank 4 out of 8 processors
Hello world from processor atl1, rank 7 out of 8 processors
Hello world from processor atl0, rank 1 out of 8 processors
Hello world from processor atl1, rank 5 out of 8 processors
Hello world from processor atl1, rank 6 out of 8 processors
------------------------------------
Begin Slurm Epilog: Oct-07-2022 16:10:11
Job ID:        1473
Array Job ID:  _4294967294
User ID:       gburdell3
Job name:      SlurmBatchMPIExample
Resources:     cpu=8,mem=8G,node=2
Rsrc Used:     cput=00:00:16,vmem=1104K,walltime=00:00:02,mem=0,energy_used=0
Partition:     ice-cpu
Nodes:         atl0,atl1
------------------------------------
```

GPU Jobs



Note

By default, your job will be assigned to the first available Nvidia GPU. If you want to use a specific Nvidia architecture, or if you wish to use an AMD GPU, you must specify the type.

Requesting GPUs

- Note that the GPU resource can be requested 2 different ways. For both approaches, the `<gpu type>` is optional, if a specific architecture is needed.
    - `--gres=gpu:<gpu type>:<number of gpus per node>` . This specifies GPUs **per node**. Note that the number provided here is for number of gpus per node.
    - `-G, --gpus=<gpu type>:<total number of gpus>` . This specifies GPUs **per job**. Note that the number provided here is for the total number of gpus. Slurm requires a minimum of 1 GPU per node, so the total number of GPUs requested must be greater than or equal to the number of nodes requested.

Examples for requesting 1 GPU:

- Nvidia Tesla V100
  - `--gres=gpu:V100:1` or `-G V100:1` for any V100
  - `--gres=gpu:1 -C V100-16GB` or `-G1 -C V100-16GB` for a V100 with 16 GB of memory
  - `--gres=gpu:1 -C V100-32GB` or `-G1 -C V100-32GB` for a V100 with 32 GB of memory
  - maximum 4 V100 per node
- Nvidia Quadro Pro RTX6000 (note underscore in some syntax)
  - `--gres=gpu:RTX_6000:1` or `-G RTX_6000:1`
  - `--gres=gpu:1 -C RTX6000` `-G 1 -C RTX6000` when using the `-C` constraint parameter
  - maximum 4 RTX6000 per node
- Nvidia A40
  - `--gres=gpu:A40:1` or `-G A40:1`
  - `--gres=gpu:1 -C A40` or `-G 1 -C A40` when using the `-C` constraint parameter
  - maximum 2 A40 per node with AMD CPUs
- Nvidia A100
  - `--gres=gpu:A100:1` or `-G A100:1` for any A100
  - `--gres=gpu:1 -C A100-40GB` or `-G 1 -C A100-40GB` for an A100 with 40 GB of memory
  - `--gres=gpu:1 -C A100-80GB` or `-G 1 -C A100-80GB` for an A100 with 80 GB of memory
  - maximum 2 A100 per node with AMD CPUs
- Nvidia H100
  - `--gres=gpu:H100:1` or `-G H100:1`
  - `--gres=gpu:1 -C H100` or `-G 1 -C H100` when using the `-C` constraint parameter
  - maximum 8 H100 per node
- Nvidia H200
  - `--gres=gpu:H200:1` or `-G H200:1`
  - `--gres=gpu:1 -C H200` or `-G 1 -C H200` when using the `-C` constraint parameter
  - maximum 8 H200 per node
- The first available NVIDIA H100 OR H200 can be requested by specifying the constraint `-C HX00` in conjunction with the `-G` or `--gres=gpus:N` flags.
- Nvidia L40S
  - `--gres=gpu:L40S:1` or `-G L40S:1`
  - `--gres=gpu:1 -C L40S` or `-G1 -C L40S` when using the `-C` constraint parameter
  - maximum 8 L40S per node
- Nvidia RTX6000 Pro Blackwell
  - `--gres=gpu:rtx_pro_6000_blackwell:1` or `-G rtx_pro_6000_blackwell:1`
  - `--gres=gpu:1 -C gpu-rtxpro-blackwell` or `-G1 -C gpu-rtxpro-blackwell` when using the `-C` constraint parameter
  - maximum 16 RTXPro Blackwells per node

    - **NOTE - These nodes have Intel Granite Rapids CPUs - for advice on compiling for this architecture, please see our guide on Compiling Programs.**
- AMD MI210
    - `--gres=gpu:MI210:1` or `-G MI210:1`
    - `--gres=gpu:1 -C MI210` or `-G 1 -C MI210` when using the `-C` constraint parameter
    - maximum 2 MI210 per node with AMD CPUs
    - For details, see Using AMD GPUs

Memory can be requested with `--mem-per-cpu` or `--mem-per-gpu`.

With Slurm, users can also take advantage of using the following variations of `--gpus*` for greater control over how GPUs are allocated:

- `--gpus-per-node=<gpu type>:<number of gpus>` - Specify the number of GPUs required for the job on each node in the job resource allocation. More information for this option can be found for salloc or sbatch.
- `--gpus-per-socket=<gpu type>:<number of gpus>` - Specify the number of GPUs required for the job on each socket in the job resource allocation. More information for this option can be found for salloc or sbatch.
- `--gpus-per-task=<gpu type>:<number of gpus>` - Specify the number of GPUs required for the job on each task in the job resource allocation. More information for this option can be found for salloc or sbatch.

Let's take a look at running a CUDA code example on H100 GPU node.

Interactive GPU Example (with an H100 GPU Node)

For running GPUs in Slurm using an interactive job with an H100 GPU node, follow the steps for Interactive Jobs to enter an interactive session:

- First, start a Slurm interactive session with an H100 GPU node with the following command, allocating for 1 node with an Nvidia H100 Tensor Core GPU.

```
[gburdell3@login-node ~]$ salloc --gres=gpu:H100:1 --ntasks-per-node=1
salloc: Pending job allocation 1234
salloc: job 1234 queued and waiting for resources
```

- Next, after pending status, your job will start after resources are granted with the following prompt:

```
salloc: job 1234 has been allocated resources
salloc: Granted job allocation 1234
salloc: Waiting for resource configuration
salloc: Nodes compute-node are ready for job
----------------------------------------
Begin Slurm Prolog: Feb-29-2024 21:35:52
Job ID:    1234
User ID:   gburdell3
Account:
Job name:  interactive
Partition: coe-gpu
----------------------------------------
[gburdell3@compute-node ~]$
```

- Next, confirm you successfully allocated an H100 GPU node by running `nvidia-smi`.

```
[gburdell3@compute-node ~]$ nvidia-smi
Thu Feb 29 21:35:58 2024
+---------------------------------------------------------------------------------------
-------+
| NVIDIA-SMI 535.86.10              Driver Version: 535.86.10    CUDA Version: 1
2.2     |
|-----------------------------------------+----------------------+----------------------
-------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncor
r. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Comp
ute M. |
|                                         |                      |
MIG M. |
|=========================================+======================+==============
=======|
|   0  NVIDIA H100 80GB HBM3          On  | 00000000:0A:00.0 Off |
     0 |
| N/A   36C    P0              72W / 700W |      4MiB / 81559MiB |      0%      D
efault |
|                                         |                      |             Di
sabled |
+-----------------------------------------+----------------------+----------------------
-------+
```

- After allocating your job, you can run commands in the interactive session, which should have `gcc` loaded by default. However, we will want to compile an example with `nvcc`, which is included with Nvidia CUDA by default:

```
[gburdell3@compute-node-1 ~]$ module load cuda
```

- Set up the environment for a simple "hello world" example with CUDA to compile using `nvcc`:

```
[gburdell3@compute-node-1 ~]$ mkdir nvcc_example
[gburdell3@compute-node-1 ~]$ cd nvcc_example
[gburdell3@compute-node-1 nvcc_example]$
```

- Create a file named `hello_cuda.cu` with the following CUDA code:

```
#include <stdio.h>

__global__ void helloCUDA()
{
    printf("Hello, world!\n");
}

int main()
{
    helloCUDA<<<1, 1>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

- Compile `hello_cuda.cu` using `nvcc` with CUDA and run the executable `hello_cuda` :

```
[gburdell3@compute-node-1 nvcc_example]$ nvcc hello_cuda.cu -o hello_cuda
[gburdell3@compute-node-1 nvcc_example]$ srun ./hello_cuda
Hello, world!
```

Batch GPU Example (with an H100 GPU Node)

For running GPUs in Slurm using a batch job with an H100 GPU node, follow the steps in Batch Jobs and Basic Python Example to set up and run a batch job:

- First, create a directory named `nvcc_example` (repeat of step from Interactive GPU Example):

```
[gburdell3@compute-node-1 ~]$ mkdir nvcc_example
[gburdell3@compute-node-1 ~]$ cd nvcc_example
[gburdell3@compute-node-1 nvcc_example]$
```

- Create a file named `hello_cuda.cu` with the following CUDA code (repeat of step from Interactive GPU Example):

```
#include <stdio.h>

__global__ void helloCUDA()
{
    printf("Hello, world!\n");
}

int main()
{
    helloCUDA<<<1, 1>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

- Next, in the `nvcc_example` directory, create a batch script named `H100_example.sbatch` script with the following content:

```
#!/bin/bash
#SBATCH -JHGX_H100_Example               # Job name
#SBATCH -N1 --ntasks-per-node=1          # Number of nodes and cores per node req
uired
#SBATCH --gres=gpu:H100:1                # GPU type (H100) and number of GPUs
#SBATCH --mem-per-gpu=224GB              # Memory per CPU core, 8 CPUs/GPU
#SBATCH -t1:00:00                        # Duration of the job (Ex: 1 hour)
#SBATCH -oReport-%j.out
#SBATCH --mail-type=BEGIN,END,FAIL       # Mail preferences
#SBATCH --mail-user=gburdell3@gatech.edu # E-mail address for notifications
cd ~/nvcc_example
module load gcc
module load cuda
nvcc hello_cuda.cu -o hello_cuda
srun ./hello_cuda
```

- Note that we recommend using `--mem-per-gpu=224GB` to allocate 224GB of memory per GPU allocated on the HGX H100 servers.
- You can run the batch file `HGX_H100_example.sbatch` by running the following command:

```
[gburdell3@compute-node-1 nvcc_example]$ sbatch H100_example.sbatch
```

- In the `nvcc_example` directory, the `Report-%j.out` output file should be generated with your job information and results:

```
----------------------------------------
Begin Slurm Prolog: Feb-29-2024 16:42:32
Job ID:    1235
User ID:   gburdell3
Account:
Job name:  HGX_H100_Example
Partition: coe-gpu
----------------------------------------
Hello, world!
----------------------------------------
Begin Slurm Epilog: Feb-29-2024 16:42:34
Job ID:        1235
Array Job ID:  _10001
User ID:       gburdell3
Account:
Job name:      HGX_H100_Example
Resources:     cpu=1,gres/gpu:h100=1,mem=224G,node=1
Rsrc Used:     cput=00:00:02,vmem=0,walltime=00:00:02,mem=928K,energy_used=0
Partition:     coe-gpu
Nodes:         compute-node-1
----------------------------------------
```

Using AMD GPUs

- The AMD GPUs can be monitored with the `rocm-smi` command.
- When compiling for these GPUs, it is essential to specify the architecture, or an error will occur. With the `hipcc` compiler, use `hipcc --offload-arch=gfx90a` .
- An example `vectoradd_hip.cpp` code can be found on AMD's site.
- `make` can be used if preferred.
- `CMake` can also be used.

Local Disk Jobs

Every ICE compute node has local disk storage available for temporary use in a job, which is automatically cleared upon job completion. Some applications can benefit from this storage for faster I/O than network storage (home and scratch). Most ICE CPU nodes and some GPU nodes have large NVMe local disks, while a few have SAS storage. See ICE resources for details.

- Use the `${TMPDIR}` variable in your Slurm script or interactive session to access the temporary directory for your job on local disk, which is automatically created for every job.
- When requesting a partial node, guarantee availability of local disk space with `#SBATCH --tmp=<size>[units, default MB]` .
- To request a node with SAS storage, add `#SBATCH -C localSAS` .
- To request a node with NVMe storage, add `#SBATCH -C localNVMe` .

Return to Main Menu

- Go back to Ice Cluster Pages
- Go back to PACE Cluster Documentation

Copy Permalink

Was this article helpful? Yes No

Related Articles

No content to display

ASC Most Useful Articles

The Significance and Purpose of Employee Performance Management Self-Assessment at Georgia Tech
KB0043151 v6.0 • Performance Management

Performance Management @Tech Probation Evaluation
KB0042087 v5.0 • Performance Management

Review/Update My Direct Deposit Information
KB0041418 v16.0 • Payroll & Taxes

How to Submit Goals in Performance Management at Tech
KB0041374 v18.0 • Performance Management

Hiring Process: Background Screenings Information for Applicants
KB0039627 v11.0 • Hiring

Georgia Institute of Technology

North Avenue
Atlanta, GA 30332

+1 404.894.2000

Campus Map

Hazing Public Disclosures

Accessibility

Accountability

Accreditation

© Georgia Institute of Technology